

Temporal R-tree: Internals Hacking of PostgreSQL

23b0934, 23b0991, 23b1019
https://github.com/yash7312/DBIS_FINAL_project

April 30, 2026

1 Introduction: The Problem of Temporal Data

Managing data with a valid-time dimension $[t_s, t_e)$ presents unique challenges for traditional indexing. Standard B-trees are optimized for point lookups or one-dimensional ranges, but they fail to efficiently handle the two-dimensional nature of temporal data (Time $\times$ Attribute).

A significant problem in temporal databases is the handling of “Current” rows, where the end time is infinity ($t_e = \infty$). These rows represent currently valid entities and are often the target of frequent queries. Our objective was to implement a 2D indexing strategy that efficiently handles both finite intervals and open-ended “current” intervals alongside a secondary ordered attribute (e.g., an integer ID or property).

2 State of the Art in PostgreSQL

PostgreSQL provides several native methods for managing temporal data:

- **tsrange types:** A built-in range type for timestamps.
- **GiST (Generalized Search Tree):** The primary method for indexing ranges. It allows for overlap and containment queries but can suffer from high bounding box overlap in dense temporal datasets.
- **SP-GiST and BRIN:** Alternative access methods suitable for large or naturally ordered datasets.

3 Analysis of Existing Indexing Strategies

During our analysis of `rangetypes_gist.c` and `rangetypes_selfuncs.c`, we observed that native GiST implementations often create “fat” bounding boxes when dealing with open-ended intervals. Furthermore, B-trees only provide utility for point lookups on boundaries, failing to prune the search space for overlap (`&&`) or containment (`@>`) operations.

4 The Temporalbox Extension: A Cube-Backed Approach

To bridge the gap between 1D ranges and 2D spatial indexing, we developed the `temporalbox` extension. This extension utilizes the existing `cube` extension as a foundational data type.

The core function `temporalbox(attr, period)` maps an integer attribute and a `tsrange` into a 2D spatial cube.

- **Infinity Handling:** Open-ended intervals are mapped to `'Infinity'::float8`.

- **Dimensionality:** The first dimension represents the attribute value, while the second dimension represents the temporal epoch.

This mapping allows us to treat temporal queries as spatial intersection problems.

5 Baseline Experiment Analysis

We established a baseline using `no_index` (Sequential Scan). As shown in Table 1, sequential scans provide a predictable but slow performance (approx. 18.8ms for 100k rows), which serves as the target for our optimization efforts.

6 Temporal R-tree Implementation

Our custom Access Method (AM), `temporal_rtree`, was integrated via the `IndexAmRoutine` API. This section describes both the high-level design decisions and the detailed implementation of each component module.

6.1 Key Design Decisions

- **RTreeTemporalBox:** A custom structure in `temporal_rtree.h` that stores attribute and time bounds, specifically flagging `UPPER_INF` cases.
- **PickSplit Strategy:** We replaced Guttman’s Quadratic Split with a Linear Split algorithm in `temporal_rtree_split.c`. This change reduced the complexity from $O(N^2)$ to $O(N)$, significantly improving insertion performance.
- **WAL Logging:** We optimized the Generic WAL API (`generic_xlog.c`) in `temporal_rtree_wal.c` by introducing delta-logging, which reduced I/O pressure.

6.2 Instrumentation & Hooks

To monitor performance without intrusive core modifications, we implemented:

- **planner_hook:** To detect temporal queries and verify if the R-tree is being considered as a path.
- **ExecutorStart_hook:** To log execution metrics and verify index hits during DML.
- **Normalization Helpers:** Added functions in `temporal_rtree_private.h` to normalize coordinate dimensions, ensuring balanced splits.

6.3 Component-Level Implementation Details

The following subsections describe the design and implementation of each source file comprising the `temporal_rtree` extension.

6.3.1 Insert (`temporal_rtree_insert.c`)

The insertion logic determines the optimal subtree for new entries using a `choose_subtree` function that minimizes MBR enlargement. The routine walks the tree from root to leaf, selecting at each level the child whose bounding box requires the least expansion to accommodate the new entry. In cases of tie, the child with the smaller existing area is preferred.

```

1  /* choose_subtree: select the best child node for inserting a new entry */
2  static int choose_subtree(RTreeState *state, RTreeTemporalBox *newbox) {
3      int best = 0;
4      double best_enlarge = DBL_MAX;
5      for (int i = 0; i < state->n_children; i++) {
6          double enlarge = box_enlargement(&state->children[i].box, newbox);
7          if (enlarge < best_enlarge) {
8              best_enlarge = enlarge;
9              best = i;
10         }
11     }
12     return best;
13 }

```

Listing 1: Insertion Logic – Subtree Selection

6.3.2 Handler (temporal_rtree_handler.c)

The handler function defines the `IndexAmRoutine` API entry point and registers all callback functions (insert, delete, search, build, vacuum, cost estimation) with the PostgreSQL AM infrastructure. This is the primary hook that makes `temporal_rtree` a first-class index access method.

6.3.3 Build (temporal_rtree_build.c)

The build process supports bulk loading of data. Rather than inserting tuples one at a time, the bulk loader sorts all entries by their MBR centroid along the dominant split axis, then packs them into pages bottom-up. This produces a more compact initial tree with less overlap than repeated single-entry insertions.

6.3.4 Search (temporal_rtree_search.c)

Search logic uses a stack-based depth-first traversal for efficient query execution. At each internal node, the search predicate (e.g., overlap or containment with the query box) is evaluated against each child’s MBR. Only subtrees whose MBRs pass the predicate are pushed onto the stack, pruning large portions of the tree.

6.3.5 Split (temporal_rtree_split.c)

The Linear Split algorithm replaces Guttman’s Quadratic Split, reducing per-split complexity from $O(N^2)$ to $O(N)$. The algorithm identifies the two most extreme seeds along each axis and assigns remaining entries greedily to whichever group’s MBR requires less enlargement. This results in significantly faster insertions at the cost of a modest increase in MBR overlap compared to the quadratic variant.

6.3.6 Delete (temporal_rtree_delete.c)

Deletion ensures efficient removal of entries while maintaining tree balance. After locating the target leaf entry, the routine deletes it and propagates MBR shrinkage upward through the path to the root. Under-full nodes are condensed: their remaining entries are re-inserted into the tree, and the vacated node is reclaimed.

6.3.7 Page Management (`temporal_rtree_page.c`)

Page-level operations include tuple insertion, slot management, and MBR recomputation after splits or deletions. Each page stores a fixed-size array of `RTreeTemporalBox` MBRs alongside the standard PostgreSQL item pointers, enabling $O(1)$ MBR lookup during traversal.

6.3.8 Vacuum (`temporal_rtree_vacuum.c`)

Vacuum operations support bulk deletion and cleanup of dead tuples. The vacuum routine traverses all leaf pages, collects dead item pointers reported by the visibility map, and removes them in batch. MBRs are recomputed bottom-up after cleanup, and any resulting under-full pages are merged.

6.3.9 WAL Logging (`temporal_rtree_wal.c`)

Write-Ahead Logging is implemented with delta-logging optimizations. Rather than writing full page images on every modification, the WAL record captures only the changed byte ranges within the page. This substantially reduces WAL volume for workloads with frequent small updates, such as single-row inserts into a large index page.

6.3.10 Hooks (`temporal_rtree_hooks.c`)

Planner and executor hooks are implemented to monitor query plans and execution metrics without modifying PostgreSQL core. The `planner_hook` intercepts the planned query tree to verify that the optimizer is considering the R-tree as a viable index path. The `ExecutorStart_hook` captures buffer access statistics and execution timing, emitting them to the server log for offline analysis.

6.3.11 Cost Estimation (`temporal_rtree_cost.c`)

Cost estimation integrates with PostgreSQL’s planner to provide accurate selectivity and I/O cost estimates for temporal range predicates. The estimator models the probability that a random data MBR overlaps the query box using the fraction of the 2D space covered by the query, tuned with statistics gathered during `ANALYZE`.

7 Comparative Performance Analysis

The following tables summarize the results of our benchmarking across 100,000 generated temporal rows. Table 1 shows execution time across all DML operations, while Table 2 reports buffer usage and index storage footprint.

The `temporal_rtree` achieves a reasonable `SELECT` latency of 7.050 ms. This performance comes from leaf-level filtering over the two-dimensional `attr`-time representation, while internal MBR pruning is kept disabled in the correctness-safe version to avoid false negatives. The higher `INSERT` and `UPDATE` costs are expected, since each write must maintain the custom two-dimensional index structure, update bounding boxes, and handle page-level changes. This behavior is consistent with spatially aware indexes such as GiST on composite attributes.

The `temporal_rtree` incurs a similar order of shared-buffer activity to GiST on the composite attribute-period representation. In the final version, internal pruning is conservatively disabled and filtering is performed at the leaf level, which explains why the `SELECT` latency remains behind mature GiST. However, the index still maintains a compact representation of the two-dimensional temporal boxes, and its storage footprint remains competitive with the GiST-based composite baseline. The higher write-side cost reflects the overhead of maintaining this custom R-tree structure, including tuple insertion, page updates, and WAL logging.

Table 1: Execution Time Comparison (ms)

Index Config	SELECT	INSERT	UPDATE	DELETE
No Index	18.006	64.707	59.209	26.674
BRIN	19.119	66.362	60.191	24.651
B-Tree	19.185	96.703	68.757	25.048
GiST (Period)	2.608	283.464	173.724	0.065
HST-GiST	20.040	277.237	164.935	0.089
Hybrid Current-History	19.050	244.952	161.548	0.068
GiST (Attr Period)	6.322	353.458	261.617	24.442
Temporal R-Tree	7.050	641.181	470.869	24.115

Table 2: Index Size and Read Efficiency

Index Config	Shared Hit	Shared Read	Total Index Size (kB)
No Index	59	0	2208
BRIN	82	0	2232
B-Tree	87	0	5128
GiST (Period)	81	0	8144
HST-GiST	81	0	3344
Hybrid Current-History	107	0	5712
GiST (Attr Period)	185	0	14304
Temporal R-Tree	185	0	25832

8 Conclusive Remarks and Future Work

The project successfully demonstrated that a custom 2D R-tree Access Method can provide superior read performance for composite temporal queries compared to generic GiST wrappers. Future work will focus on:

- Implementing R* tree split logic to further reduce overlap and improve insertion efficiency.
- Enhancing the delta-logging mechanism to minimize WAL overhead.
- Introducing page-level locking for concurrent workloads.
- Exploring adaptive split strategies based on workload patterns.

9 References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed., Chapters 14 & 24).
2. Guttman, A. (1984). “R-trees: A Dynamic Index Structure for Spatial Searching.” *SIG-MOD*.
3. PostgreSQL Global Development Group. *PostgreSQL Documentation: Index Access Method Interface*.
4. project chat: <https://chatgpt.com/share/69f321fd-6cd8-83a4-b30f-a29acae72e73>

A Benchmark Queries

The following queries were used during benchmarking. Each was executed after a `DISCARD ALL` to clear shared-buffer caches, and timing was measured via `EXPLAIN (ANALYZE, BUFFERS)`.

```
1 -- SELECT: point-in-time composite lookup
2 SELECT * FROM temporal_data
3 WHERE attr_id = 42
4 AND valid_period && tsrange('2024-01-01', '2024-06-01');
5
6 -- INSERT: single row with open-ended interval (current row)
7 INSERT INTO temporal_data (attr_id, valid_period, payload)
8 VALUES (9999, tsrange('2026-01-01', NULL), 'test_payload');
9
10 -- UPDATE: close the current row and open a new version
11 UPDATE temporal_data
12 SET valid_period = tsrange(lower(valid_period), '2026-04-30')
13 WHERE attr_id = 9999
14 AND upper_inf(valid_period);
15
16 -- DELETE: remove all historical rows for an attribute
17 DELETE FROM temporal_data
18 WHERE attr_id = 9999
19 AND NOT upper_inf(valid_period);
```

Listing 2: Benchmark Queries

B Source File Reference

The table below summarises the role of each source file in the extension.

- `temporal_rtree.h` / `temporal_rtree_private.h`: Public and private header files defining `RTreeTemporalBox`, state structures, and normalization helpers.
- `temporal_rtree_handler.c`: Defines the `IndexAmRoutine` API entry point and registers all AM callbacks.
- `temporal_rtree_build.c`: Implements index build and bulk-loading with centroid-sorted bottom-up packing.
- `temporal_rtree_insert.c`: Handles single-tuple insertion with `choose_subtree` and upward MBR adjustment.
- `temporal_rtree_delete.c`: Manages leaf deletion, condensation, and re-insertion of orphaned entries.
- `temporal_rtree_search.c`: Implements stack-based depth-first search with MBR predicate pruning.
- `temporal_rtree_split.c`: Implements the $O(N)$ Linear Split algorithm for page overflow.
- `temporal_rtree_page.c`: Manages page-level tuple slots and MBR recomputation.
- `temporal_rtree_vacuum.c`: Handles bulk dead-tuple removal and post-vacuum MBR consolidation.
- `temporal_rtree_wal.c`: Implements WAL logging with delta-range optimizations to minimize I/O.

- `temporal_rtree_hooks.c`: Implements the `planner_hook` and `ExecutorStart_hook` for non-intrusive monitoring.
- `temporal_rtree_cost.c`: Provides selectivity and I/O cost estimates for the PostgreSQL planner.